

Construction Replay Transcript

EDI 278 Prior Auth — Design · 2026-08-13 · what you'll hear in documents/construction-replay.mp4

What this interface does

Ingest X12 278 prior-auth requests, apply completeness rules (member, procedure, diagnosis, attachment when required), simulate a payer decision, and emit an X12 278 response plus an EHR HL7 ORU "auth decided" notice. Demo only — not Da Vinci PAS, not real payer APIs.

Who it's for

- Sources: Provider PA drop folder (input/inbound/*.edi)
- Destinations: decision buckets (approved / denied / incomplete / pending), output/responses/*.edi (278), output/ehr-notices/*.hl7 (ORU), SQL AuthRequest status
- Demo vs production: Demo only — synthetic 278, catalog rules in SQL, no FHIR PAS, no LLP to a live EHR
- Anyone watching the construction video or reading this transcript

Systems involved

- Sources: Provider PA drop folder (input/inbound/*.edi)
- Destinations: decision buckets (approved / denied / incomplete / pending), output/responses/*.edi (278), output/ehr-notices/*.hl7 (ORU), SQL AuthRequest status
- Demo vs production: Demo only — synthetic 278, catalog rules in SQL, no FHIR PAS, no LLP to a live EHR

Documents we also produce

EDI278_Prior_Auth_Capability_Brief.pdf · ready

Short overview of what the interface does — shareable without reading route XML.

EDI278_Prior_Auth_Test_Plan.pdf · ready

Scenarios that prove the interface works end-to-end.

Module documentation pack (15 deep-dive PDFs) · ready

PilotFish product PDFs for the modules used in this build.

EDI278_Prior_Auth_V2_Route_Diagrams.pdf · ready

V2 route diagrams for visual review of module topology.

construction-replay.mp4 + this transcript · generate when ready

Narrated walkthrough of building the routes, then a live Demo-tab test when inject is available.

Narration

Spoken lines from the construction video (setup, build-replay, live test, closing).

Setup

Welcome

Welcome to the PilotFish demo: EDI 278 Prior Auth. Ingest X12 278 prior-auth requests, apply completeness rules (member, procedure, diagnosis, attachment when required), simulate a payer decision, and emit an X12 278 response plus an EHR HL7 ORU "auth decided" notice. PilotFish supports the whole flow out of the box.

Pipeline overview

Before we wire anything, here's the flow end to end. Ingest X12 278 prior-auth requests, apply completeness rules (member, procedure, diagnosis, attachment when required), simulate a payer decision, and emit an X12 278 response plus an EHR HL7 ORU "auth decided" notice.

External systems

And here's what sits around the routes. SQL Server, PilotFish eiPlatform, and Demo Web UI. All of those are spun up in docker images for this demo.

1 - Poll And Score Auths

First route

Ingest X12 278 prior-auth requests, apply completeness rules (member, procedure, diagnosis, attachment when required), simulate a payer decision, and emit an X12 278 response plus an EHR HL7 ORU "auth decided" notice. We'll build that in 2 routes. Starting with a blank canvas on the first route.

Poll EDI 278 Files

Here's a directory listener — it watches a local folder. That folder is /opt/pilotfish/input/inbound. We're only taking .edi,278.txt files. It checks about every 5 seconds. After pickup we move the file aside so the folder stays clean.

Fork - XPath

This XPath fork splits the file so each transaction set becomes its own message.

Extract 278 Auth Fields

XPath evaluation copies a few fields off the XML onto the transaction so later steps can use them.

Debug Write Forked Transaction

This writes a debug copy of the XML so we can inspect what the route saw. That lands under /opt/pilotfish/output/debug.

Lookup Auth Catalog

This is a SQL lookup — we query Edi278PriorAuth over JDBC using a key from the transaction.

Extract Catalog Fields

XPath evaluation copies a few fields off the XML onto the transaction so later steps can use them.

Compose Auth Patient Name

Then we stitch last name and first name into one patient name on the transaction.

Build Auth Decision XML

Now for the mapping — we're using the stock XSLT processor with build-auth-decision.xslt. It maps AuthTraceNumber, MemberId, PatientLast.

Debug Write Auth Decision

This writes a debug copy of the XML so we can inspect what the route saw. That lands under /opt/pilotfish/output/debug.

Pretty-Print Auth Decision XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Stage Auth Decision

Then we write the file out to /opt/pilotfish/output/staged-decisions.

2 - Route Auth Decisions

Next route

Next route — blank canvas again, then the modules one at a time.

Staged-folder listener

Here's a directory listener — it watches a local folder. That folder is /opt/pilotfish/output/staged-decisions. We're only taking .xml files. It checks about every 3 seconds.

Extract Decision Attributes

XPath evaluation copies the match bucket and underpay flag off the decision XML so the router can fan out.

Conditional Router

Then a conditional router — it uses XPath to pick which path this transaction takes.

Map Auth Decision To 278 XCSDData

Now for the mapping — we're using the stock XSLT processor with build-278-response.xslt. It maps trace, d, hcr.

Map Auth Decision To ORU XML

Now for the mapping — we're using the stock XSLT processor with build-oru-notice.xslt. It maps trace, d, disp.

Update Auth Request Status

Then we update that row in Edi278PriorAuth so the status matches the decision.

Pretty-Print Approved XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Pretty-Print Denied XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Pretty-Print Incomplete XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Pretty-Print Pended XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

XML to EDI 278 Response

Here's the EDI transformation — it turns our EDI XML into X12 text on the wire.

XML to HL7 ORU

Here's a HL7 Transformation processor — next step in the flow.

Update Auth Request

The SQL update already ran on this path, so we finish on a null transport — nothing else to send.

Write Approved

Then we write the file out to /opt/pilotfish/output/approved.

Write Denied

Then we write the file out to /opt/pilotfish/output/denied.

Write Incomplete

Then we write the file out to /opt/pilotfish/output/incomplete.

Write Pended

Then we write the file out to /opt/pilotfish/output/pended.

Write 278 Response

Then we write the file out to /opt/pilotfish/output/responses.

Normalize ORU Line Endings

Here's an End of Line processor — next step in the flow.

Write ORU Notice

Then we write the file out to /opt/pilotfish/output/ehr-notices.

Live test

After construction, the video switches to the Demo tab and runs a live inject when the UI supports it.

Prove it works

Routes are built. Let's switch to the Demo tab and prove it works.

Submit a sample

I'll submit this sample from the Demo tab — same as an operator would.

Waiting for the routes

Give the routes a moment to pick up the file and write the output.

AuthRequest

There it is — output on the results panel.

Submit another sample

I'll send it one more time so we can watch it land again.

Waiting for the routes

Same path — the second result should show up here.

AuthRequest

And there's the second result.

Closing

Demo complete

That's the demo — ingest X12 278 prior-auth requests, apply completeness rules (member, procedure, diagnosis, attachment when required), simulate a payer decision, and emit an. Thanks for choosing PilotFish.